

电子科技大学

计算机专业类课程

实验报

课程名称：计算机网络编程

学 院：计算机学院

专 业：人工智能

学生姓名

指导教师：向渝

日 期： 2025 年 12 月 14 日

电子科技大学

实验报告

实验三

一、实验名称：针对 TIME 服务的 UDP 客户端软件的实现

二、实验学时：4

三、实验内容和目的：

掌握客户端软件的工作原理。

掌握针对 TIME 服务的 UDP 客户端软件的编程实现方法

四、实验原理：

使用指定编程工具编写针对 TIME 服务的 UDP 客户端软件的源代码。

使用指定编程工具对代码进行编译和调试，生成执行程序。

在 PC 机上执行程序，检验程序访问 TIME 服务的功能。

五、实验步骤：

1.总体设计思路

本实验采用 C 语言基于 UDP 协议，实现一个针对 TIME 服务（端口 37）的客户端程序。客户端通过 UDP 套接字向 TIME 服务器发送空数据报，请求当前时间，并接收服务器返回的 32 位二进制时间值，再将其转换为 Unix 时间并输出本地可读时间。

程序整体流程为：

创建 UDP 套接字 → 解析服务器地址 → 发送时间请求 → 接收时间响应

→ 时间格式转换 → 输出结果

2.创建 UDP 套接字

客户端调用 `socket()` 创建一个基于 IPv4 的 UDP 套接字 (`SOCK_DGRAM`), 用于与 TIME 服务器进行无连接通信。若套接字创建失败, 程序立即输出错误信息并终止执行。

```
sockfd=socket(AF_INET,SOCK_DGRAM,0);
if(sockfd<0){
    error("Failed to create socket\n");
}
```

3 解析 TIME 服务器地址

程序通过 `gethostbyname()` 函数将命令行参数中给出的服务器主机名或 IP 地址解析为网络地址, 并将解析结果保存至 `hostent` 结构体中, 用于后续服务器地址结构体的初始化。

```
server=gethostbyname(argv[1]);
if(server==NULL){
    fprintf(stderr,"Cant explain host:%s\n",argv[1]);
    exit(EXIT_FAILURE);
}
```

4.服务器地址结构体初始化

使用 `sockaddr_in` 结构体保存 TIME 服务器的地址信息, 设置地址族为 `AF_INET`, 端口号为 TIME 服务的标准端口 37, 并将解析得到的 IP 地址复制到结构体中, 确保网络通信参数正确。

```
//init server address struct
memset(&server_addr,0,sizeof(server_addr));
server_addr.sin_family=AF_INET;
memcpy(&server_addr.sin_addr.s_addr,server->h_addr,server->h_length);
server_addr.sin_port=htons(TIME_SERVER_PORT);
if(sendto(sockfd,buffer,0,0,(struct sockaddr *)&server_addr,sizeof(server_addr))<0){
    error("Fail to send data");
}
```

5.发送 TIME 请求报文

根据 TIME 协议的规定, 客户端无需发送实际数据, 只需向服务器发送一个空的 UDP 数据报即可触发时间响应。程序通过 `sendto()` 向 TIME 服务器发送长度为 0 的数据报, 完成时间请求。

```
//send empty data to TIME
socklen_t server_len=sizeof(server_addr);
ssize_t n=recvfrom(sockfd,buffer,BUFFER_SIZE,0,(struct sockaddr *)&server_addr,
&server_len);
if(n<0){
    error("Fail to receive data");
}else if(n!=BUFFER_SIZE){
    fprintf(stderr,"Receive wrong data length:%zd\n",n);
    exit(EXIT_FAILURE);
}
```

6.接收 TIME 服务器响应

客户端调用 `recvfrom()` 接收 TIME 服务器返回的数据。程序对接收数据的长度进行判断，确保返回的数据长度为 4 字节，以符合 TIME 协议返回 32 位时间值的规范。

```
//send empty data to TIME
socklen_t server_len=sizeof(server_addr);
ssize_t n=recvfrom(sockfd,buffer,BUFFER_SIZE,0,(struct sockaddr *)&server_addr,
&server_len);
if(n<0){
    error("Fail to receive data");
}else if(n!=BUFFER_SIZE){
    fprintf(stderr,"Receive wrong data length:%zd\n",n);
    exit(EXIT_FAILURE);
}
```

7.时间数据转换与结果输出

程序将接收到的 4 字节二进制数据转换为 32 位无符号整数，并使用 `ntohl()` 将其从网络字节序转换为主机字节序。随后，程序减去 TIME 协议与 Unix 纪元之间的时间差，将其转换为 Unix 时间戳，并输出对应的本地时间。

```
//Convert the received binary data into a 32-bit unsigned integer
memcpy(&time_val,buffer,sizeof(time_val));
time_val=ntohl(time_val);

const unsigned int UNIX_EPOCH_DIFF=2208988800U;
if(time_val<UNIX_EPOCH_DIFF){
    fprintf(stderr,"Invalid time\n");
    exit(EXIT_FAILURE);
}
current_time=time_val-UNIX_EPOCH_DIFF;
printf("Receive from TIME server %s:%u\n",argv[1],time_val);
printf("Convert to Unix time:%ld\n", (long)current_time);
printf("Local time:%s",ctime(&current_time));
close(sockfd);
return EXIT_SUCCESS;
```

六、实验数据及结果分析：

1.程序运行与 TIME 服务响应分析

编译并运行 TIME 客户端程序后，客户端成功向指定 TIME 服务器发送请求，并接收到服务器返回的 4 字节时间数据。程序未出现阻塞或超时现象，说明 UDP 通信过程正常完成。



```
yar@2023080910002:~/computer_network$ ./time_client_udp time.nist.gov
Receive from TIME server time.nist.gov:3974691723
```

2.接收数据格式正确性分析

实验中，客户端接收到的数据长度为 4 字节，符合 TIME 协议规定的时间数据格式要求。若接收到的数据长度异常，程序能够及时检测并给出错误提

示，增强了程序的健壮性。

```
yar@2023080910002:~/computer_network$ ./time_client_udp time.nist.gov  
Receive from TIME server time.nist.gov:3974691723
```

3.时间戳转换结果分析

客户端将 TIME 服务器返回的时间值转换为 Unix 时间戳，并输出对应的本地时间字符串。输出结果与系统当前时间基本一致（存在网络传输及服务器时钟差异），验证了时间转换逻辑的正确性。

```
Convert to Unix time:1765702923  
Local time:Sun Dec 14 17:02:03 2025
```

4.UDP 通信特性分析

本实验采用 UDP 协议进行通信，客户端无需建立连接即可完成时间请求与响应。实验结果表明，在数据量小、实时性要求较高的应用场景下，UDP 协议能够提供高效、简洁的通信方式。